

LA MÉMOIRE VIRTUELLE

UVCI

Table des matières



Objectifs	3
Introduction	4
I - CONCEPT DE MEMOIRE VIRTUELLE	5
1. Concept et définition	5
2. Principes et fonctionnement	5
II - Évaluation 1	7
III - LA PAGINATION A LA DEMANDE	9
1. Principe	9
2. Défaut de page	9
IV - Évaluation 2	11
V - REMPLACEMENT DE PAGE	13
1. Principe	13
2. Les algorithmes de remplacement de page	13
VI - Évaluation 3	15
VII - Les algorithmes de remplacement de page et leur fonctionnement.	17
1. Algorithme FIFO	17
2. Algorithme optimal	18
3. Algorithme LRU	18
4. Algorithmes basés sur le comptage	18
5. Tirage Aléatoire	19
6. Algorithme de la seconde chance	19
7. Allocation de cadres de pages :	19
VIII - Évaluation 4	20

Objectifs

À la fin de cette leçon, vous serez capable de :

- Décrire et expliquer le concept de mémoire virtuel
- Expliquer la méthode de pagination à la demande
- Expliquer la notion de remplacement de page
- Identifier et décrire les algorithmes de remplacement de page et leur fonctionnement

Introduction



Les techniques de gestion de la mémoire vue précédemment possèdent le même objectif : maintenir plusieurs processus en mémoire simultanément afin de permettre la multiprogrammation. Cependant, la mémoire centrale est limitée et peut être très vite saturée. Que faut-il donc faire en cas d'insuffisance de mémoire vive ?

La mémoire virtuelle est une technique permettant d'exécuter un programme partiellement chargé en mémoire centrale ; Le principal avantage de ce schéma est que les programmes peuvent être plus grands que la mémoire physique (mémoire centrale).

CONCEPT DE MEMOIRE VIRTUELLE

I

1. Concept et définition

Le concept de mémoire virtuelle a été mis au point après que l'on a constaté que dans de nombreux cas, on n'a pas besoin d'un programme dans sa totalité.

Ainsi la possibilité d'exécuter un programme se trouvant partiellement en mémoire présenterait plusieurs bénéfices :

- Un programme n'est plus limité par la quantité de mémoire physique disponible. Les utilisateurs pourront écrire des programmes pour un espace adresse virtuel extrêmement grand, simplifiant ainsi la tâche de programmation.
- Comme chaque programme utilisateur pourrait occuper moins de mémoire physique, il serait possible d'exécuter plus de programme en même temps.

La mémoire virtuelle est une partie du disque dur ou d'un support de stockage qu'on prend et dans laquelle on stocke une partie des processus en cours d'exécution, lorsqu'il y a insuffisance de mémoire vive.

La mémoire virtuelle permet :

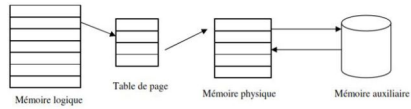
- d'utiliser de la mémoire de masse comme extension de la mémoire vive ;
- d'augmenter le taux de multiprogrammation ;
- de mettre en place des mécanismes de protection de la mémoire ;
- de partager la mémoire entre processus.

2. Principes et fonctionnement

- *Fondamental*

La mémoire virtuelle repose sur le principe suivant : la taille de l'ensemble formé par le programme, les données et la pile peut dépasser la capacité disponible de mémoire physique. Le système d'exploitation conserve les parties de programme en cours d'utilisation dans la mémoire principale, et le reste sur le disque.

La mémoire virtuelle fournit donc un espace d'adressage extrêmement grand alors que la mémoire physique est limitée. Cela est possible en utilisant une mémoire auxiliaire comme espace de travail pour charger et décharger les différentes pages par le SE.



Mémoire virtuelle plus grande que la mémoire physique

- *Remarque : Mémoire virtuelle et ES*

Quand un programme attend le chargement d'une partie de lui-même, il est en attente d'E/S et ne peut s'exécuter; le CPU peut par conséquent être donné à un autre processus, de la même façon que pour tout autre système préprogrammé.

- *Méthode : Implémentation de la mémoire virtuelle*

Il existe deux techniques permettant d'implanter une mémoire virtuelle. Les "overlays" et la pagination à la demande. La pagination à la demande est la plus performante et la plus utilisée et fera l'objet de notre étude.

Évaluation 1


II

Exercice

La mémoire virtuelle est :

☐ Une technique permettant d'exécuter des programmes virtuels

☐

Une technique utilisée par le système exploitation pour exécuter les processus en cas d'insuffisance de la mémoire vive

☐

Une technique utilisée par le système exploitation pour exécuter des processus en cas de défaillance de la mémoire centrale

☐

Une technique utilisée par le système exploitation pour exécuter des processus en cas de fragmentation de la mémoire physique

☐

Une technique utilisée par le système exploitation pour exécuter des processus en cas de défragmentation de la mémoire principale

Exercice

La mémoire virtuelle est :

☐

Une zone de la mémoire vive qu'on prend et dans laquelle on stocke une partie des processus en cours d'exécution

☐

Une zone de la mémoire ROM qu'on prend et dans laquelle on stocke une partie des processus en cours d'exécution

☐

Une zone du disque dur où d'une mémoire de stockage qu'on prend et dans laquelle on stocke une partie des processus en cours d'exécution

☐ Une mémoire logicielle créée virtuellement à partir d'un programme

☐ Un ensemble de techniques pour virtualiser les données, programmes et mémoire d'un ordinateur

Exercice

La mémoire virtuelle permet :

- ☐ de se passer d'une mémoire vive pour exécuter des applications
- ☐ d'augmenter la taille de la mémoire vive
- ☐ de rendre plus rapide l'exécution d'un processus
- ☐ d'exécuter un programme plus volumineux que la mémoire centrale

Exercice

Les avantages de la mémoire virtuelle sont : (4 réponses)

- ☐ Partage de mémoire entre processus.
- ☐ Mise en place des mécanismes de protection de la mémoire ;
- ☐ Partage de PCB entre processus
- ☐ Augmentation du taux de multiprogrammation ;
- ☐ Partage des fragment de mémoire de la mémoire vive entre processus
- ☐ Utilisation de la mémoire de masse comme extension de la mémoire vive ;
- ☐ Réduction de la taille des processus
- ☐ Réduction de la durée d'exécution d'un processus

Exercice

Les deux techniques permettant d'implanter une mémoire virtuelle sont :

- ☐ Les overlays
- ☐ Les mémoire siamoises
- ☐ La pagination à la demande.
- ☐ Les mémoire fixe contiguës
- ☐ Les mémoire dynamiques contiguës

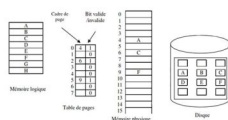
LA PAGINATION A LA DEMANDE

III

1. Principe

La mémoire virtuelle est communément implémentée avec la pagination à la demande. Quand un processus doit être transféré en mémoire, la routine de pagination devine quelles pages seront utilisées avant que le processus soit mis en mémoire auxiliaire de nouveau. Au lieu de transférer en mémoire un processus complet, la routine de pagination ramène seulement les pages qui lui sont nécessaires. Ainsi, elle évite que l'on charge en mémoire des pages qui ne seront jamais employées, en réduisant le temps de swaping (va-et-vient) et la quantité de mémoire dont on a besoin.

Avec cette technique, le SE doit disposer de moyens pour distinguer les pages qui sont en mémoire, et celles qui sont sur disque. Par exemple, on peut utiliser dans la table des pages un bit valide/invalide pour décrire si la page est chargée en mémoire ou non.



Mémoire paginée avec certaines pages en mémoire

L'algorithme suivant résume la technique de pagination à la demande.

```

le programme avec une adresse logique paginée (p)
SI p n'est valide ALORS
    générer l'adresse physique
FINON
/* Défaut de page */
lire p de la mémoire auxiliaire /* swap in */
SI p n'est dans libre ou mémoire centrale ALORS
    générer l'adresse physique
    mettre à jour la table de page
    générer l'adresse physique
FINON
/* algorithme de remplacement */
choisir une page présente en mémoire centrale
l'écrire en mémoire auxiliaire /* swap out */
mettre à jour le bit de validité
affecter la case libre à la nouvelle page
et mettre à jour la table de page
générer l'adresse physique
FIN

```

2. Défaut de page

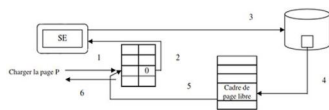
Que se passe-t-il si un processus essaie d'utiliser une page qui n'est pas en mémoire ? L'accès à une page marquée invalide provoque un défaut de page : La MMU remarque que la page est absente (ce qui est indiqué par le bit de présence) et fait procéder le CPU à un déroutement, c'est-à-dire que le processeur est restitué au système d'exploitation.

La MMU (Memory Management Unit) fait correspondre les adresses logiques à des adresses physiques en se basant sur une table de page.

La procédure permettant de traiter ce défaut de page est la suivante :

- S'assurer que la référence de la page est correcte.
- S'assurer que la page désirée est bien en mémoire auxiliaire.
- Trouver un cadre de page libre et charger la page.

- le système d'exploitation sélectionne une case peu utilisée et écrit son contenu sur le disque;
 - il transfère ensuite la page qui vient d'être référencée dans la case libérée,
 - modifie la correspondance
- Recommence l'instruction déroutée.



Procédure de traitement d'un défaut de page

Légende :

- 1 : on fait référence à la page P qui n'existe pas en mémoire.
- 2 : Il y a un déroutement vers le SE.
- 3 : Le SE vérifie si la page existe bien en mémoire auxiliaire.
- 4 : Le SE vérifie s'il y a un cadre de page libre, auquel cas il y charge la page absente.
- 5 : Le SE met à jour la table de page.
- 6 : Redémarrer l'instruction du processus qui a provoqué le défaut de page.

Évaluation 2

IV

Exercice

La pagination à la demande consiste :

- ☐ A stocker les pages d'un processus en mémoire cache et à les exécuter la la demande du processus
- ☐

A stocker les pages d'un processus sur le disque et à ne ramener en mémoire vive que celles qui sont nécessaire.

☐

A exécuter les instructions se trouvant dans une page en mémoire centrale lorsqu'elle est demandée par un processus.

- ☐ A paginer une page à la demande d'un processus.

Exercice

Le MMU (Memory Management Unit) permet :

- ☐ De fait correspondre les adresses virtuelles à des adresses physiques par le biais de ma mémoire cache
- ☐ De fait correspondre les adresses virtuelle à des adresses logique, en se basant sur une table de segment
- ☐ De fait correspondre les adresses logiques à des adresses physiques, en se basant sur une table de page ;
- ☐

Permet une gestion efficace de la mémoire par allocation dynamique de données dans un environnement de processus concurrentiel.

Exercice

Un défaut de page est :

☐

Un phénomène qui se produit lorsque le processeur accède à une page ne se trouvant pas en mémoire centrale.

- ☐ Un phénomène qui se produit lorsque deux processus se trouvent en interblocage
- ☐

Un phénomène qui se produit lorsque le processeur accède à une page ne se trouvant pas en mémoire virtuelle.

- ☐ Un phénomène qui se produit lorsque qu'un processus est dérouté vers une autre zone mémoire.

Exercice

La procédure permettant de traiter un défaut de page est :

- ☐ S'assurer que la référence de la page est correcte.
- ☐ S'assurer que la page désirée est bien en mémoire centrale.
- ☐ S'assurer que la page désirée est bien en mémoire auxiliaire.
- ☐ Trouver un cadre de page libre en mémoire logique et charger la page.
- ☐ Trouver un cadre de page libre centrale et charger la page.
- ☐ Recommence l'instruction déroutée.
- ☐ Trouver un cadre de page libre en mémoire virtuelle et charger la page.
- ☐ Trouver un cadre de page libre en mémoire et supprimer la page.

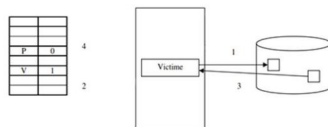
REPLACEMENT DE PAGE

V

1. Principe

Lorsque le SE se rend compte au moment de charger une page qu'il n'existe aucun cadre de page disponible, il peut faire recours à un remplacement de page. Ainsi le code complet d'une procédure de traitement d'un défaut de pages est le suivant :

1. Trouver l'emplacement de la page désirée sur disque.
2. Trouver un cadre de page libre. S'il existe un cadre de pages libre, l'utiliser, sinon utiliser un algorithme de remplacement de pages pour sélectionner un cadre de page victime.
3. Enregistrer la page victime dans le disque et modifier la table de page.
4. Charger la page désirée dans le cadre de page récemment libéré et modifier la table de pages.
5. Redémarrer le processus utilisateur.



Légende :

- 1 : Swap out de la page victime.
- 2 : Mettre le bit de la page à invalide
- 3 : Swap in de la page désirée.
- 4 : Mettre le bit de la page à valide

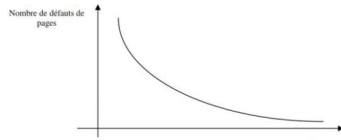
2. Les algorithmes de remplacement de page

Il existe plusieurs algorithmes différents de remplacement de pages. En général, on souhaite celui qui provoquera le taux de défauts de pages le plus bas.

On évalue un algorithme en l'exécutant sur une séquence particulière de références mémoires et en calculant le nombre de défauts de page. Cette chaîne est appelée : chaîne de références.

Afin de déterminer le nombre de défauts de pages pour une chaîne de références et un algorithme de remplacement particulier, on doit également connaître le nombre de cadres de pages disponibles.

Evidemment, au fur et à mesure que le nombre de cadres de pages augmente, le nombre de défauts de pages doit diminuer, comme le montre la figure suivante :



- Remplacement de page (local / global)

Lors d'un défaut de page, s'il n'y a pas de case libre pour charger la page demandée comment choisir la page à remplacer ? Un algorithme de remplacement peut être :

- Local : si sa victime est choisie parmi les pages allouées au programme
- Global : si sa victime est choisie parmi l'ensemble de toutes les pages présentes en mémoire centrale.

Définition : Chaîne de références

On appelle chaîne de références la séquence des numéros de pages référencées par un programme durant une exécution. Un bon algorithme de remplacement doit diminuer le nombre de défauts de pages engendrés par une chaîne de référence définie par l'exécution d'un processus.

Évaluation 3

VI

Exercice

On appelle chaîne de référence :

- ☐ la séquence des numéros de pages référencées par un ensemble de programmes durant leurs exécutions
- ☐

une suite continue et discrète d'instructions se trouvant au sein d'un processus et analyser puis exécuter durant la durée de vie d'un processus

- ☐ une chaîne linéaire et adiabatique d'instructions stockées en mémoire vive
- ☐ la séquence des numéros de pages référencées par un programme durant une exécution

Exercice

On appelle algorithme de remplacement de page :

☐

Un algorithme permettant de choisir une page victime à remplacer en mémoire centrale en cas de défaut de page.

- ☐ Un algorithme permettant de remplacer les pages obsolètes en mémoire centrale

☐

Un algorithme permettant de choisir une page victime à remplacer en mémoire centrale en cas de surcharge de la mémoire.

☐

Un algorithme permettant de choisir une page victime à remplacer en mémoire centrale en cas de surcharge du processeur

☐

Un algorithme permettant de choisir des pages victimes à remplacer en mémoire en cas de suppression ou de mise à jour d'un document en mémoire.

Exercice

On dit qu'un algorithme de remplacement de page est local :

- ☐ si la page victime est choisie parmi les pages allouées au programme

- ☐ si la page victime est choisie dans la mémoire cache du processeur
- ☐ si la page victime est choisie dans les registres du processeur
- ☐ si sa victime est choisie parmi l'ensemble de toutes les pages présentes en mémoire centrale
- ☐ si la page victime est choisie parmi les pages se trouvant en mémoire virtuelle

Exercice

On dit qu'un algorithme de remplacement de page est global :

- ☐ si sa victime est choisie parmi l'ensemble de toutes les pages présentes en mémoire centrale
- ☐ Si la page victime se trouve en mémoire auxiliaire
- ☐ si la page victime est choisie dans les registres du processeur
- ☐ si la page victime est choisie dans la mémoire cache du processeur
- ☐ Si la page victime se trouve sur un serveur externe au que le processus

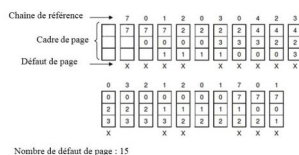
Les algorithmes de remplacement de page et leur fonctionnement.

1. Algorithme FIFO

L'algorithme de remplacement FIFO (First In First Out) est le plus simple à réaliser. Avec cet algorithme, quand on doit remplacer une page, c'est la plus ancienne qu'on doit sélectionner. Son principal intérêt est sa simplicité de réalisation : il suffit d'entretenir dans une file FIFO les numéros des cases où sont chargées les pages successives.

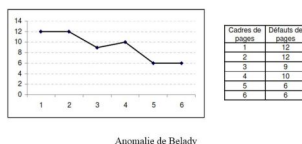
Exemple : Considérons un système à mémoire paginée ayant 3 cadres de pages (pages physiques), et soit la chaîne de références suivante : 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1. Quel est le nombre de défauts de pages si l'algorithme de remplacement choisi est l'algorithme FIFO ?

Pour répondre à cette question, faisons un déroulement qui fait ressortir les états successifs du système et en marquant les différents défauts de pages.



L'algorithme de remplacement FIFO est simple à implémenter mais ses performances ne sont pas toujours bonnes.

Afin d'illustrer les problèmes rencontrés avec un algorithme de remplacement FIFO, on peut envisager la chaîne de références suivantes : 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5. La figure suivante montre le nombre de défauts de pages en fonction des cadres de pages disponibles.



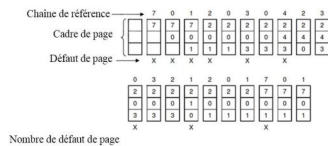
Anomalie de Belady

On remarque que le nombre de défauts de pages pour 4 cadres de pages (ie : 10) est supérieur au nombre de défauts de pages pour 3 cadres de pages (ie : 9) !. Ce résultat fort inattendu est connu sous le nom de l'anomalie de Belady. Elle décrit une situation où le taux de défauts de pages augmente au fur et à mesure que le nombre de cadres de pages augmente, contrairement à la règle générale.

2. Algorithme optimal

L'algorithme de remplacement optimal fournit le taux de défauts de pages le plus bas de tous les autres algorithmes, de plus il ne souffre pas de l'anomalie de Belady. Son principe est de remplacer la page qui mettra le plus de temps à être de nouveau utilisée.

Exemple : Reprenons la même chaîne de références que l'algorithme FIFO précédent, et calculons le nombre de défauts de pages avec cet algorithme.

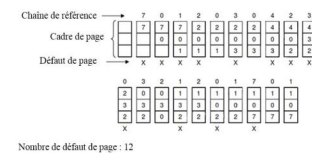


Malheureusement, l'algorithme optimal est difficile à mettre en œuvre car il requiert une connaissance future de la chaîne de références. Il est utilisé essentiellement pour faire des études comparatives.

3. Algorithme LRU

L'algorithme LRU (Least Recently Used) sélectionne pour le remplacement d'une page victime, la page la moins récemment utilisée. C'est à dire qu'on doit remplacer une page, l'algorithme sélectionne la page qui n'a pas été utilisée pendant la plus grande période de temps.

Exemple : Reprenons la même chaîne de références que l'algorithme FIFO précédent, et calculons le nombre de défauts de pages avec cet algorithme.



4. Algorithmes basés sur le comptage

Il existe de nombreux algorithmes de remplacement de pages basés sur le comptage du nombre de références effectuées à chaque page. Pour cela, on utilise un compteur pour chaque numéro de page.

- L'algorithme LFU :

L'algorithme de remplacement LFU (Least Frequently Used) requiert que l'on remplace la page la moins fréquemment utilisée ; c'est à dire celle ayant le compteur le plus petit. La raison de cette sélection est qu'une page activement utilisée devrait avoir un grand compte de références.

Exemple : En reprenant la même chaîne de références que l'algorithme FIFO précédent, le nombre de défauts de pages trouvé avec cet algorithme est de 11.

Cet algorithme présente un problème quand une page est très utilisée pendant la phase initiale d'un programme, mais qu'elle n'est plus employée à nouveau par la suite. Comme elle a été amplement utilisée, elle possède un grand compte et reste donc en mémoire même si elle n'est pas utilisée.

- *L'algorithme MFU :*

L'algorithme de remplacement MFU (Most Frequently Used) requiert que l'on remplace la page la plus fréquemment utilisée. La raison de cette sélection est que la page ayant le compte le plus petit vient probablement d'être ramenée en mémoire et doit encore être utilisée.

Exemple : En reprenant la même chaîne de références que l'algorithme FIFO précédent, le nombre de défauts de pages trouvé avec cet algorithme est de 14.

5. Tirage Aléatoire

La victime est choisie au hasard (loi uniforme) parmi l'ensemble des pages présentes en mémoire. Cet algorithme n'a aucune vertu particulière, car il ne tient aucun compte du comportement observé ou prévisible du programme; il sert surtout de point de comparaison.

6. Algorithme de la seconde chance

Cet algorithme est une approximation très grossière de LRU. A chaque case est associé un bit d'utilisation U, mis à 1 lors d'une référence à la page qu'elle contient. Les cases sont ordonnées dans une file circulaire et un pointeur ptr désigne la dernière case chargée. Le pointeur progresse jusqu'à la première case dont le bit U est à zéro; les cases rencontrées en route, dont le bit U est à 1, reçoivent une seconde chance (de n'être pas prises comme victime) et leur bit U est mis à 0. Cet algorithme est également appelé algorithme de l'horloge "Clock", la progression du pointeur étant analogue à celle de l'aiguille d'une horloge. La mise à 1 du bit U lors d'une référence peut être réalisée par un mécanisme matériel ou logiciel.

7. Allocation de cadres de pages :

Comment allouer la quantité de mémoire libre entre les différents processus ? Par exemple, si on dispose de 93 cadres de pages libres et 2 processus, combien de cadres de pages obtient chacun d'eux ?

La manière la plus simple de répartir m cadres de page entre n processus est de donner à chacun des portions égales, c'est à dire m/n cadres de pages. Par exemple, si on possède 93 cadres de pages et 5 processus, chaque processus obtiendra 18 cadres de pages. Les 3 cadres de pages restant pourraient faire partie d'un pool de buffers de cadres de pages libres. Ce schéma est appelé allocation équitable.

Il existe une autre méthode qui admet que les processus ont des besoins différents et demandent des quantités différentes de mémoires. Par exemple, nous disposons d'une mémoire composée de 62 cadres de pages et de deux processus différents : un processus utilisateur qui demande 10 Ko et un processus de Bases de données qui demande 127 Ko.

Cette seconde méthode utilise l'allocation proportionnelle, c'est à dire qu'elle affecte la mémoire disponible entre les processus en fonction de leur besoin. Soit S la taille de la mémoire virtuelle d'un processus P_i . On a :

$$S = \sum S_i$$

Si le nombre total de cadres de pages disponibles est m, on alloue a_i cadres de pages au processus P_i , où :

$$a_i = (s_i/S) * m$$

Ainsi, le nombre de cadres de pages affectés à chaque processus de notre exemple est :

- Processus utilisateur : $(10/137) * 62 = 4$ cadres de pages.
- Processus Base de données : $(127/137) * 62 = 57$ cadres de pages.

Évaluation 4

VIII

Exercice

Quatre algorithmes de remplacement de page sont :

- ☐ L'algorithme de Belady
- ☐ L'algorithme de sage Makora
- ☐ L'algorithme LFU
- ☐ L'algorithme des sages de Sion
- ☐ L'algorithme select medium
- ☐ L'algorithme de tourniquet
- ☐ L'algorithme LRU
- ☐ L'algorithme optimal
- ☐ L'algorithme SRF
- ☐ L'algorithme de remplacement FIFO

Exercice

L'anomalie de Belady est un phénomène qui :

☐

Se produit lorsque le taux de défaut de page diminue au fur et à mesure que le nombre de cadre de page augmente.

☐

décrit une situation où le taux de défauts de pages augmente au fur et à mesure que le nombre de cadres de pages augmente.

☐

se produit lorsque la mémoire atteint un état de saturation ou aucun processus ne peut être chargé en mémoire.

☐ Se produit lorsque la mémoire virtuelle atteint son état maximal.

☐

Décrit une situation où deux processus veulent accéder simultanément à une même page en mémoire virtuelle.

Exercice

L'algorithme de remplacement de page FIFO permet de choisir comme page victime :

- ☐ La page la moins récemment utilisée
- ☐ La page la moins fréquemment utilisée
- ☐ La page se trouvant au sommet de la pile en mémoire centrale
- ☐ La page qui mettra plus de temps à être de nouveau utilisée
- ☐ La page la plus ancienne en mémoire centrale

Exercice

L'algorithme de remplacement de page FIFO permet de choisir comme page victime :

- ☐ La page la moins récemment utilisée
- ☐ La page la moins fréquemment utilisée
- ☐ La page se trouvant au sommet de la pile en mémoire centrale
- ☐ La page qui mettra plus de temps à être de nouveau utilisée
- ☐ La page la plus ancienne en mémoire centrale

Exercice

L'algorithme de remplacement de page LRU permet de choisir comme page victime :

- ☐ La page la moins récemment utilisée
- ☐ La page la moins fréquemment utilisée
- ☐ La page se trouvant au sommet de la pile en mémoire centrale
- ☐ La page qui mettra plus de temps à être de nouveau utilisée
- ☐ La page la plus ancienne en mémoire centrale